path-sum of the log odds of the second model winning, over the entirety of $\mathcal{G}(m)$:

$$s(\theta)_m = \frac{1}{|\mathcal{G}(m)|} \sum_{g \in \mathcal{G}(m)} \left(\log \frac{\theta'((1, g_{1,1}))}{1 - \theta'((1, g_{1,1}))} + \sum_{a \in g} \log \frac{\theta'(a)}{1 - \theta'(a)} \right), \tag{12}$$

where $\theta'(a) = \theta(a)\mathbb{1}\{a \in \mathcal{A}\} + (1 - \theta((a_2, a_1)))\mathbb{1}\{a \notin \mathcal{A}\}$, with the convention that $\theta((m, m)) = 1/2$ for all m. Note that for any $g \in \mathcal{G}(m)$ where $a \in g$ and $m \notin a$, we also have some $g' \in \mathcal{G}(m)$ such that $(a_2, a_1) \in g$. Meanwhile, if $a \in g$ and $m \in a$, then $a = (m', m)$ for some m'. Thus, we can compute

$$s(\theta)_m = \sum_{\substack{a \in \mathcal{A} \\ m \notin a}} \frac{1}{2} \left(\log \frac{\theta'(a)}{1 - \theta'(a)} + \log \frac{\theta'((a_2, a_1))}{1 - \theta'((a_2, a_1))} \right) + \sum_{m' \in [M] \setminus \{m\}} \left(\log \frac{\theta'((m', m))}{1 - \theta'((m', m))} + \frac{\theta'((1, m'))}{1 - \theta'((1, m'))} \right) \tag{13}$$

$$= \sum_{\substack{a \in \mathcal{A} \\ m \notin a}} \frac{1}{2} \left(\log \frac{\theta(a)}{1 - \theta(a)} + \log \frac{1 - \theta(a)}{\theta(a)} \right) + \sum_{m' \in [M] \setminus \{m\}} \left(\log \frac{\theta'((m', m))}{1 - \theta'((m', m))} + \frac{\theta'((1, m'))}{1 - \theta'((1, m'))} \right) \tag{14}$$

$$= \sum_{m' \in [M] \setminus \{m\}} \left(\log \frac{\theta'((m', m))}{1 - \theta'((m', m))} + \frac{\theta'((1, m'))}{1 - \theta'((1, m'))} \right) \tag{15}$$

$$= \sum_{m' \in [M] \setminus \{m\}} \left((1 - 2\mathbb{1}\{m' > m\}) \log \frac{\theta((m', m))}{1 - \theta((m', m))} + \frac{\theta((1, m'))}{1 - \theta((1, m'))} \right). \tag{16}$$

This score is always well-defined, and is a simple, smooth function of θ. Its derivative is, for all $a \in \mathcal{A}$,

$$\frac{\partial}{\partial \theta(a)} s(\theta)_m = \mathbb{1}\{a_2 = m\}(1 - 2\mathbb{1}\{a_1 > m\}) \frac{1}{\theta(a)(1 - \theta(a))} + \mathbb{1}\{a_1 = 1,\ a_2 \neq m\} \frac{1}{\theta(a)(1 - \theta(a))}. \tag{17}$$

How is the BT score related to the original Bradley-Terry model? In the original Bradley-Terry model, $H_t \in \{0, 1\}$, and the probability of model m beating model m' is assumed to be given by

$$\theta((m', m)) = \frac{e^{\xi_m}}{e^{\xi_m} + e^{\xi_{m'}}}, \tag{18}$$

for some unknown parameters $\xi_1, \ldots, \xi_M$—the *Bradley-Terry coefficients*. The basic goal of the Bradley-Terry model is to estimate these parameters from the observed outcomes. In our setting, however, we use the outcomes to get a CLT on θ, and then can immediately recover the coefficients. Taking without loss of generality $\xi_1 = 0$, we have that

$$\log \frac{\theta((1, m'))}{1 - \theta((1, m'))} + \log \frac{\theta((m', m))}{1 - \theta((m', m))} = \log \frac{\theta((1, m'))}{\theta((m', 1))} + \log \frac{\theta((m', m))}{\theta((m, m'))} \tag{19}$$

$$= \log \frac{e^{\xi_{m'}}(e^{\xi_{m'}} + 1)}{e^{\xi_{m'}} + 1} + \log \frac{e^{\xi_m}(e^{\xi_{m'}} + e^{\xi_m})}{e^{\xi_{m'}}(e^{\xi_{m'}} + e^{\xi_m})} \tag{20}$$

$$= \xi_{m'} + \xi_m - \xi_{m'} = \xi_m \tag{21}$$

Thus, all the sums over paths in (12) are equal to $\xi_m - \xi_{g_{1,1}}$.

$$\log \frac{\theta'((1, g_{1,1}))}{1 - \theta'((1, g_{1,1}))} + \sum_{a \in g} \log \frac{\theta'(a)}{1 - \theta'(a)} \tag{22}$$

$$= \xi_{g_{1,1}} + \xi_{g_{1,2}} - \xi_{g_{1,1}} + \xi_{g_{2,2}} - \xi_{g_{2,1}} + \cdots + \xi_{g_{M-1,2}} - \xi_{g_{M-1,1}} \tag{23}$$

$$= \xi_{g_{M-1,2}} = \xi_m. \tag{24}$$

Thus, if the parametric BT model is well-specified, the nonparametric version will exactly recover the Bradley-Terry coefficients. However, our nonparametric analogue of the BT model has major advantages over the original: it will retain statistical validity even if H_t is not binary, if the win rate is non-transitive, and if the logistic model assumed by the BT model is misspecified. In practice, the nonparametric BT coefficient can be easily computed by (16).

C. Valid P-Value

Consider the p-value

$$p_i = \frac{1}{|\mathcal{H}_{A'_i}| + 1} \left(1 + \sum_{h \in \mathcal{H}_{A'_i}} \mathbb{1}\left\{ h \geq H'_i \right\} \right). \tag{25}$$

We will prove that this p-value is valid, i.e., that $\mathbb{P}(p_i \leq t) \leq t$, under the null hypothesis that the vector $\mathcal{H}' = (H_t : A_t = A'_i) \| (H'_i)$ is exchangeable, where $\|$ denotes the concatenation operator. First, notice that we can equivalently write

$$p_i = \frac{1}{|\mathcal{H}'|} \sum_{h \in \mathcal{H}'} \mathbb{1}\left\{ h \geq H'_i \right\}. \tag{26}$$

We also have that $\mathbb{P}(p_i \leq t) \leq \mathbb{P}(p_i \leq \frac{k}{|\mathcal{H}'|})$, where $k = \lfloor t|\mathcal{H}'| \rfloor / |\mathcal{H}'|$. Then, since the data points are exchangeable, we have that H'_i is uniformly distributed among the ranks of $\mathcal{H}'$, so $\mathbb{P}(p_i \leq \frac{k}{|\mathcal{H}'|}) \leq \frac{k}{|\mathcal{H}'|} \leq t$, completing the proof.

D. Sample Prompts

D.1. Prompt Examples From Different Clusters

Example 1: An example user prompt from the cluster "Python Game Programming Challenge". This prompt distinctively separates GPT-4-0613 performance from Llama-2-70b-chat. The user request "higher quality code“ from each assistant. While GPT-4-Turbo provides a correct implementation along with helpful explanation, Llama-2-70b-chat fail to provide any program and provided a less helpful response overall.

<|User Prompt|>

Develop a snowman game similar to the Hangman game but draws a snowman figure standing on the snow instead of a hangman. It also downloads (or uses library) the complete English dictionary to select words for three levels of difficulty. Think step-by-step, take a deep breath and choose a higher quality code for each step.

<|GPT-4-0613's Answer|>

Sure, here's a step-by-step guide on how to create a Snowman game similar to Hangman. We'll use Python for this example.

Step 1: Import necessary libraries

```python
import random
import urllib.request
```

Step 2: Download the English dictionary

```python
url = "https://raw.githubusercontent.com/dwyl/english-words/master/words.txt"
urllib.request.urlretrieve(url, 'words.txt')
```

Step 3: Load the words from the dictionary and categorize them into three levels of difficulty

```python
with open('words.txt') as word_file:
    english_words = word_file.read().split()
easy_words = [word for word in english_words if 4 <= len(word) <= 6]
medium_words = [word for word in english_words if 7 <= len(word) <= 9]
hard_words = [word for word in english_words if len(word) >= 10]
```

Step 4: Create a function to randomly select a word based on the difficulty level

```python
def get_random_word(difficulty):
    if difficulty == 'easy':
        word = random.choice(easy_words)
    elif difficulty == 'medium':
        word = random.choice(medium_words)
    else:
        word = random.choice(hard_words)
    return word.upper()
```

Step 5: Create a function to print the snowman